Day 1 - Full Stack Fundamentals

Introduction to Full Stack Development using Python

Prepared by: Sarthak Jejurkar | Deliverable: PDF + Architecture Diagram

Topics Covered

No.	Topic
1	Full Stack Development - definition
2	Frontend, Backend and Database
3	Client-Server architecture
4	HTTP request-response flow
5	REST API explanation
6	Python Full Stack technology stack
7	Real-world application example

1. What is Full Stack Development?

Full Stack Development is the practice of building **both sides of a web application**: the part users see and interact with (the **frontend**) and the part that runs behind the scenes (the **backend**, including the **database**). A **Full Stack Developer** can design the user interface, write the server logic, manage data storage, and deploy the complete product.

The word *stack* refers to the layers of technology that work together to deliver one application, from the screen the user clicks on, down to the data stored on disk.

2. Frontend, Backend and Database

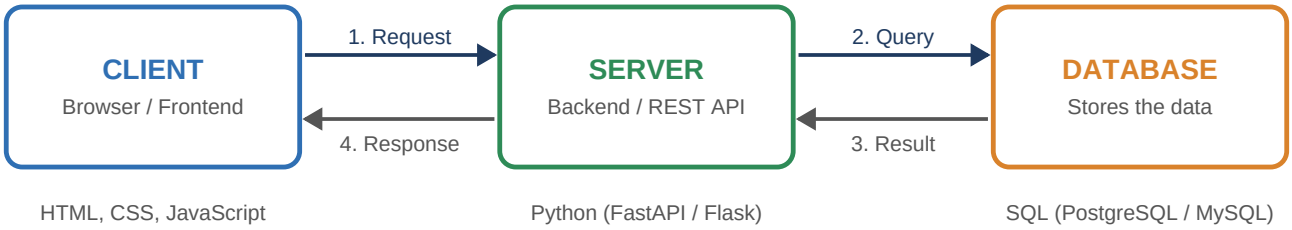
Layer	Role	Examples
Frontend (Client side)	Everything the user sees and interacts with: pages, forms, buttons, charts. Runs in the browser.	HTML, CSS, JavaScript, ReactJS
Backend (Server side)	Handles business logic, authentication, validation and exposes APIs. Receives requests and sends responses.	Python (FastAPI, Flask, Django), Node.js
Database	Stores, retrieves and organizes application data permanently and securely.	PostgreSQL, MySQL, SQLite, MongoDB

Frontend vs Backend: the frontend focuses on *presentation and user experience*; the backend focuses on *logic, security and data handling*. Neither works alone, since the frontend needs the backend to fetch and save data, and the backend needs the frontend to give users a way in.

3. Client-Server Architecture

Web applications follow the **client-server model**. The **client** (usually a browser) asks for something, and the **server** processes the request, talks to the **database** if needed, and sends back a result.

Client-Server Architecture



Key points: the client never talks to the database directly; the server acts as a secure middle layer. One server can serve thousands of clients at the same time.

4. HTTP Request-Response Flow

Communication between client and server happens over **HTTP** (HyperText Transfer Protocol). Every interaction is a **request** from the client followed by a **response** from the server.

Step-by-step cycle

- The user performs an action (opens a page, submits a form).
- The browser builds an **HTTP request** with a method, URL, headers and optional body.
- The server receives it, validates it and runs the backend logic.
- If data is required, the backend queries the **database**.
- The server sends an **HTTP response** with a status code and data (usually JSON).
- The browser reads the response and updates the screen.

Example

```
REQUEST
GET /api/products/101 HTTP/1.1
Host: example.com
Accept: application/json

RESPONSE
HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 101, "name": "Laptop", "price": 55000 }
```

Common status codes

Code	Meaning
200 OK	Request succeeded
201 Created	New resource created
400 Bad Request	Invalid data sent by client
401 / 403	Not authenticated / not allowed
404 Not Found	Resource does not exist
500 Server Error	Something failed on the server

5. REST API Explanation

REST (Representational State Transfer) is a set of rules for designing APIs. An **API** (Application Programming Interface) is the bridge that lets the frontend and backend exchange data. A REST API treats everything as a **resource** (users, products, orders) identified by a URL, and uses standard HTTP methods to act on it. Data is usually exchanged as **JSON**.

HTTP Method	Action (CRUD)	Example Endpoint
GET	Read data	GET /api/users
POST	Create new data	POST /api/users
PUT / PATCH	Update existing data	PUT /api/users/5
DELETE	Remove data	DELETE /api/users/5

REST principles: client-server separation, **stateless** requests (each request carries all information needed), uniform URL structure, and resource-based design.

6. Python Full Stack Technology Stack

Layer	Technologies
Frontend	HTML, CSS, JavaScript, ReactJS (optional: Bootstrap / Tailwind)
Backend	Python with FastAPI, Flask or Django
Database	PostgreSQL / MySQL / SQLite; ORM such as SQLAlchemy
API	REST APIs exchanging JSON
Tools	Git and GitHub, Postman, VS Code
Deployment	Render, Railway, AWS, Docker

7. Real-World Application Example: Online Shopping Website

Consider what happens when a user opens a product page on an e-commerce site:

Step	What happens	Layer
1	User clicks on a product in the browser.	Frontend
2	Browser sends GET /api/products/101 to the server.	HTTP request
3	Python backend (FastAPI) receives and validates the request.	Backend
4	Backend runs SELECT * FROM products WHERE id = 101.	Database
5	Server returns the product details as JSON with status 200.	HTTP response
6	Frontend renders name, price and image on screen.	Frontend

Summary: Full Stack Development combines frontend, backend and database into one working application. The client and server communicate using HTTP requests and responses, usually through REST APIs. A Full Stack Developer designs the UI, builds the API, manages data and deploys the product, and with Python this workflow becomes clean and beginner friendly.